Docker Storage (Definition)

- By default, all files created inside a container are stored on a writable container layer. This means that the data doesn't persist when that container no longer exists.
- Docker has two options for containers to store files on the host machine, so that the files are persisted even after the container stops: **volumes** and **bind mounts**.
- The third option supports containers storing files in-memory on the host machine using **tmpfs**.

Docker Storage (Commands)

command	description
`docker volume create`	Create a volume
`docker volume ls`	List volumes
`docker volume rm`	Remove one or more volumes
`docker volume inspect`	Display detailed information on one or more volumes

Volume examples

```
# create a volume named my_volume_1:
docker volume create my_volume_1

# list all volumes:
docker volume ls

# list all information about the volume my_volume_1:
docker volume inspect my_volume_1

# remove the volume named my_volume_1:
docker volume rm my_volume_1

# run a container from mysql image and mount the volume my_volume_2 to /var/lib/mysql/ on the container:
docker container run \
        -d
        -e MYSQL_ROOT_PASSWORD=pass123 \
        --mount type=volume,src=my_volume_2,target=/var/lib/mysql/
        mysql

# run a container from mysql image and mount the host path /etc/mydir to /var/lib/mysql/ on the container:
docker container run \
        -d
        -e MYSQL_ROOT_PASSWORD=pass123 \
        --mount type=bind,src=/etc/mydir/,target=/var/lib/mysql/
        mysql

# run a container from mysql image use the tmpfs as mount to /var/lib/mysql/ on the container:
docker container run \
        -d
        -e MYSQL_ROOT_PASSWORD=pass123 \
        --mount type=tmpfs,target=/var/lib/mysql/
        mysql
```

Docker storage (Demo)

- Play with volume storage commands (ls/create/inspect/rm)
- Create a new volume
- Mount the volume to a container named container_1 on /data/
- Mount the volume to a container named container_2 on /data/
- Create a file in /data/ inside each container with the container's hostname
- Mount the volume to a container named container_3 and run "ls /data/", you should see two files

Docker storage (Notes)

volumes	bind mounts	tmpfs
A Docker volume is mounted into a container.	A file or directory on the host machine is mounted into a container.	the container can create files in the host memory.
Docker space	Host space	Memory space of the host
/var/lib/docker/volumes	/path/to/dir/ Or /path/to/file	In-memory
Managed by Docker CLI commands Or the Docker API	-	-
Persistent	Persistent	Non-persistent
Strong portability	Weak portability	Data is destroyed with the container

Docker networks (Definition)

- Container networking refers to the ability for containers to connect to and communicate with each other, or to non-Docker workloads.
- Containers have networking enabled by default, and they can make outgoing connections.
- You can create custom, user-defined networks, and connect multiple containers to the same network.
- Once connected to a user-defined network, containers can communicate with each other using container **IP addresses** or container **names**.

Docker networks (Commands)

command	description
`docker network create`	Create a network
`docker network ls`	List networks
`docker network rm`	Remove one or more networks
`docker network inspect`	Display detailed information on one or more networks
`docker network connect`	Connect a container to a network
`docker network disconnect`	Disconnect a container from a network

Network examples

```
# create a network named my_network_1:
docker network create my_network_1

# list all networks:
docker network ls

# list all information about the network named my_network_1:
docker network inspect my_container_1

# remove the network named my_network_1:
docker network rm my_network_1

# create a network named my_network_2 and set its subnet as 182.17.0.0/16:
docker network create --driver bridge --subnet 182.17.0.0/16 my_network_2
```

Docker networks (Demo)

- Play with network commands (ls/create/rm/inspect/…)
- Play with options:
  - --driver <network-driver-name>
  - --subnet <subnet-CIDR-range>
- Additional options **for container command**:
  - -p <host-port>:<container-port>
  - --network <network-name>

Docker networks (Notes)

Null (None)	host	Bridge (default)
Isolate a container from the host and other containers.	Remove isolation between the container and the Docker host.	Connect containers on the same bridge, while providing isolation from containers that aren't connected to that bridge.
No IP	No IP (but uses the host IP)	IP is allocated from the network’s CIDR
-	Can connect with other containers using their Ips (acts as the host machine)	Can connect with neighbor containers using their IPs or names

Docker Compose (Definition)

- Docker Compose is a tool that allows you to define and run multi-container Docker applications in a single file. It simplifies the process of defining, managing, and scaling multi-container Docker applications.
- A Docker Compose file is written in YAML format and typically named “docker-compose.yml”

Docker compose (Commands)

command	description
`docker compose up`	Builds, creates, starts, and attaches to containers for a service.
`docker compose down`	Stops containers and removes containers, networks, volumes, and images created by **`docker compose up`**.
`docker compose ls`	Lists running Compose projects
`docker compose restart`	Restarts all stopped and running services, or the specified services only.

Compose examples

```yaml
version: 3.8 # Specifies the Compose file syntax version.
services: # Defines the containers that make up your application.
  web: # Service configs: Includes image, ports, volumes, networks, etc.
    build: ./path/to/app-code/ # Builds an image and runs a container from it
    ports:
      - 80:3000
    networks:
      - frontend
      - fullstack
    volumes:
      - ./html/:/var/www/html/
  db:
    image: mysql:5.7
    environment:
      MYSQL_ROOT_PASSWORD: password
    networks:
      - backend
      - fullstack
    volumes:
      - db_data:/var/lib/mysql
networks: Defines/creates the networks which the services will use.
  frontend: # Creates a new network.
  backend: # Creates a new network.
  fullstack:
    external: true # Uses an existing network.
volumes: # Defines/creates persistent data storage for containers.
  db_data: # Creates a new volume.
```

Docker compose (Demo)

- Play with compose commands (up/down/ls/…)
- Use a service that builds and uses a new image from a local Dockerfile
- Include network, storage, variable configurations

Docker compose (Notes)

- A network is created and used by default if not explicitly provided by user.
- User can create new networks belonging to the compose project or use existing networks.
- User can create new volumes belonging to the compose project or use existing volumes.
- User can build new images to be used in the compose project or use existing images.
- Project name can be overridden.